

KAZ



MOTORS

Aplicación de gestión para
concesionario de motocicletas

TFG 2025/26

Antonio Jesús Castillo Henares

IES Gran Capitán · Córdoba

BACKEND

ANDROID

REST API

DOCKER

JWT

OBJETIVOS DEL PROYECTO

Objetivo general

Desarrollar una aplicación Android nativa que integre la gestión completa de un concesionario de motocicletas y su taller, ofreciendo una plataforma única para clientes, mecánicos, vendedores y administradores, con backend REST desplegado en Docker.

Objetivos específicos

1

Catálogo multimedia / Motos nuevas

Implementar un catálogo de motos nuevas con galería de imágenes, vista 360° interactiva y reproducción de vídeo.

2

Gestión de segunda mano

Permitir a vendedores publicar motos de ocasión y a clientes buscarlas y reservarlas con filtro tolerante a errores tipográficos.

3

Módulo de taller

Desarrollar el ciclo completo de citas: solicitud, aceptación, presupuesto, aprobación, entrega y cierre.

4

Sistema de roles

Implementar 5 roles diferenciados (Admin, Vendedor, Mecánico, Cliente, Invitado) con permisos divididos por endpoint.

5

Seguridad

Asegurar la API con JWT stateless y BCrypt, incluyendo confirmación de email y reset de contraseña.

6

Despliegue

Contenerizar el backend con Docker Compose y exponerlo con HTTPS mediante ngrok sin necesidad de servidor privado virtual.

OBJETIVOS DEL PROYECTO

1

Diseñar el modelo de datos: entidades como motocicletas, marcas, clientes, ventas y citas.

2

Implementar una base de datos relacional que garantice la integridad de los datos.

3

Desarrollar un backend con Spring Boot que proporcione una API REST para gestionar la información.

4

Implementar operaciones CRUD para motos, citas, reservas, clientes, vendedores y mecánicos.

5

Desarrollar una aplicación móvil Android que interactúe con la API REST.

6

Integrar la app Android con el backend mediante consumo de servicios web.

7

Implementar funcionalidades de consulta del catálogo, registro de clientes y gestión de ventas.

8

Implementar un sistema de gestión de citas para el taller: nuevas citas de revisión o reparación.

9

Permitir la modificación de citas existentes: fecha, hora u otra información.

10

Permitir la cancelación o eliminación de citas para mantener la agenda actualizada.

11

Gestionar la disponibilidad de mecánicos y permitir asignación en paralelo.

12

Evitar la asignación de citas cuando no haya mecánicos disponibles.

13

Usar contenedores Docker para desplegar la base de datos y facilitar la portabilidad.

14

Implementar búsqueda y filtrado de motocicletas por marca, modelo, precio u otras características.

15

Gestionar imágenes de motocicletas: almacenar y visualizar fotografías desde la app.

16

Creación de usuarios con roles diferenciados: Invitado, Cliente, Administrador, Vendedor y Mecánico.

ANÁLISIS Y REQUISITOS

REQUISITOS FUNCIONALES

RF01	El sistema permitirá el registro de clientes con confirmación por email.
RF02	El sistema soportará inicio de sesión con JWT y 5 roles diferenciados.
RF03	Los clientes podrán añadir vehículos propios a su perfil.
RF04	Los clientes podrán solicitar citas de taller con fecha y hora.
RF05	Los mecánicos podrán crear y enviar presupuestos al cliente.
RF06	Los clientes podrán aceptar o rechazar el presupuesto recibido.
RF07	Los vendedores podrán publicar motos nuevas con galería, 360° y vídeo.
RF08	Los clientes podrán reservar motos de segunda mano.
RF09	El admin podrá crear cuentas de mecánicos y vendedores.
RF10	El sistema enviará notificaciones en bandeja cuando la moto esté lista.

REQUISITOS NO FUNCIONALES

RNF01	Seguridad Contraseñas con BCrypt. Comunicación HTTPS. Tokens JWT con caducidad.
RNF02	Rendimiento Carga de imágenes con Glide y precarga de frames 360° para evitar latencia.
RNF03	Escalabilidad Backend stateless. Docker Compose permite añadir instancias.
RNF04	Usabilidad App con navegación intuitiva, feedback de errores con Toast y gestos nativos.
RNF05	Compatibilidad Android minSdk 26 (Android 8.0+). Backend Java 21.
RNF06	Mantenibilidad Arquitectura MVVM en Android y capas Controller-Service-Repository en backend.

ÍNDICE

1 Descripción del proyecto

2 Arquitectura del sistema

3 Tecnologías utilizadas

4 Seguridad: JWT + BCrypt

5 Modelo de datos

6 Roles y funcionalidades

7 App Android — Pantallas

8 Funcionalidades destacadas

9 Despliegue

10 Conclusión

1 · DESCRIPCIÓN DEL PROYECTO

¿Qué es Kaz Motors?

Aplicación Android nativa que integra un concesionario de motocicletas y un taller mecánico en una sola plataforma. Permite a los clientes consultar el catálogo, reservar motos de segunda mano y gestionar sus citas de taller. Los empleados gestionan el inventario, las reparaciones y los presupuestos desde la misma app.

Catálogo de motos

Nuevas con galería, vista 360° e vídeo. Segunda mano con búsqueda tolerante a errores tipográficos.

Taller y citas

Solicitud de citas, presupuestos detallados con IVA y ciclo completo de estados hasta entrega.

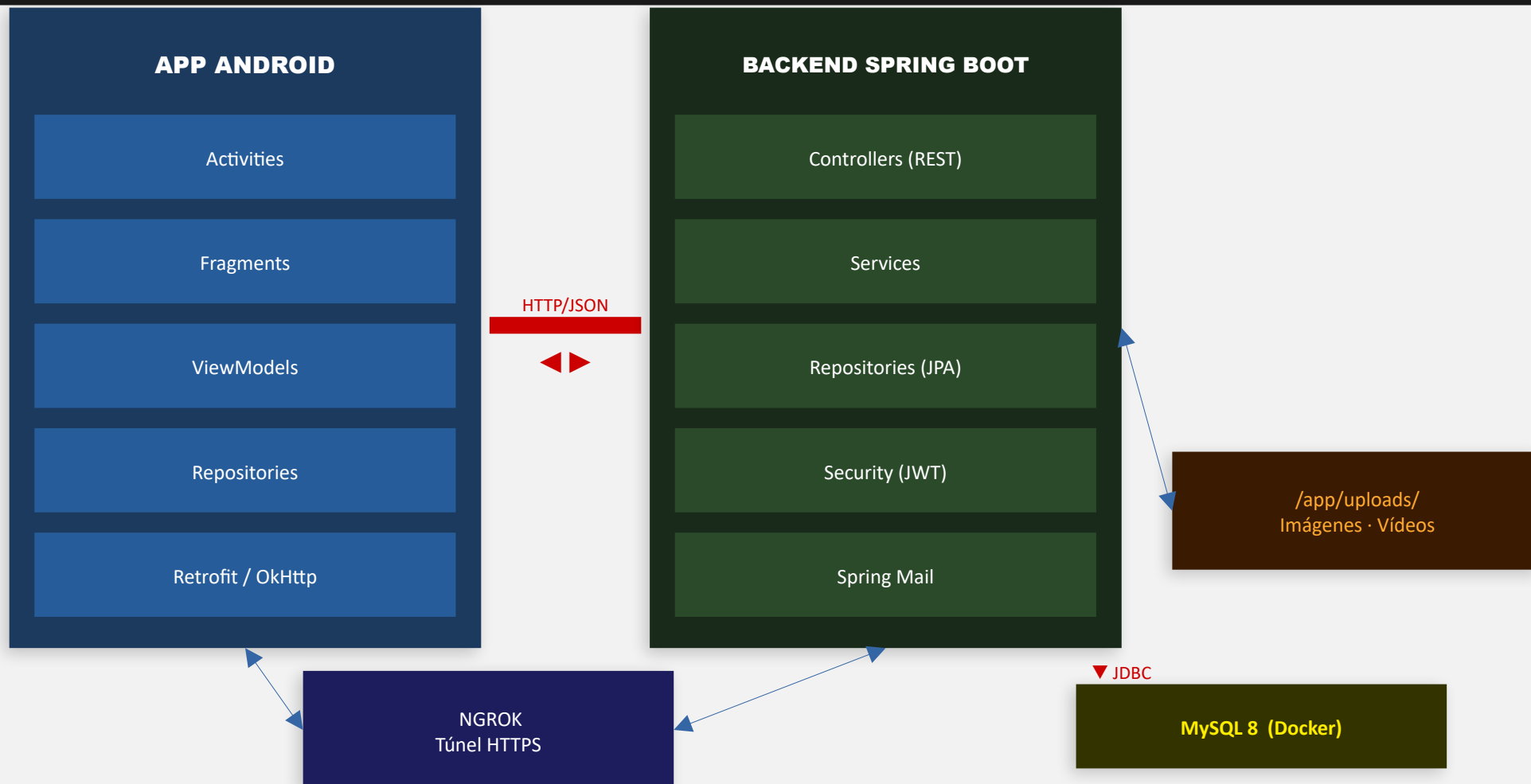
Múltiples roles

5 roles diferenciados: Cliente, Mecánico, Vendedor, Admin e Invitado, cada uno con su flujo.

Backend REST

Spring Boot 3 + Kotlin, autenticación JWT, emails de confirmación y reset de contraseña.

2 · ARQUITECTURA DEL SISTEMA



3 - TECNOLOGÍAS UTILIZADAS

BACKEND



Spring Security

JWT + BCrypt

Spring Mail

Emails automáticos

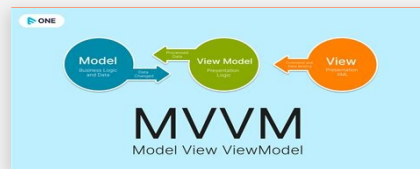


ANDROID

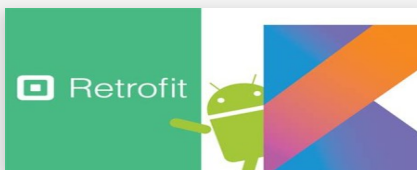
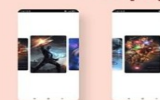


ExoPlayer

Reproducción vídeo



Auto Infinite Slider with ViewPager2



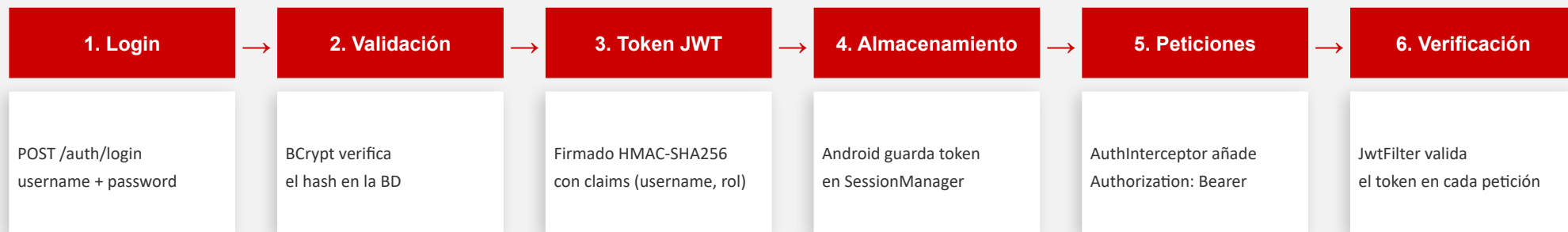
Material 3

Diseño UI



4 · SEGURIDAD — JWT + BCrypt

Flujo de autenticación



¿Por qué JWT y BCrypt?

Stateless

El servidor no guarda sesiones. Cada petición se autentica con el token, lo que permite escalar horizontalmente.

Claims embebidos

El token incluye username y rol, evitando consultas extra a la base de datos en cada petición.

BCrypt

Hash adaptativo con sal aleatoria. Aunque la BD quede comprometida, las contraseñas son inviables de descifrar por fuerza bruta.

Caducidad

El token tiene tiempo de vida limitado. El reset de contraseña usa tokens UUID con caducidad de 30 minutos.

5 - Modelo de datos — Usuarios y Roles

1 / 3



Rol (ENUM): ADMIN · VENDEDOR · MECANICO · CLIENTE

Cada usuario tiene exactamente un registro en la tabla de su rol (relación 1:1).

5 - Modelo de datos — Módulo Taller

2 / 3

MOTO_CLIENTE		1:N	CITA		1:1	REPARACIÓN	
matricula	PK		id	PK		id	PK
marca			cliente_id	FK → CLIENTE		cita_id	FK → CITA
modelo			moto	FK → MOTO_CLIENTE		descripción	
km	INT		mecanico_id	FK → MECANICO		total	DECIMAL
cliente_id	FK → CLIENTE		fecha / hora			estado	ENUM
			tipo	ENUM		fecha	
			estado	ENUM			

Cita.tipo: REVISION · MANTENIMIENTO

Cita.estado: PENDIENTE · ACEPTADA · RECHAZADA · CANCELADA · COMPLETADA

Rep.estado: ENVIADO · TALLER · RECHAZADO · RECOGER · COMPLETADO

MOTO_NUEVA		1:N	IMG_MOTO_NUEVA		MOTO_2a_MANO		1:N	RESERVA	
id	PK		id	PK	id	PK		id	PK
marca			motoNueva_id	FK	matricula			cliente_id	FK → CLIENTE
modelo			url		marca			motoSegunda_id	FK
categoria	ENUM		tipo	GALERIA / R360	modelo			fecha	
anio	INT		orden	INT	precio	DECIMAL		estado	ENUM
precio	DECIMAL				cilindrada	INT			
cilindrada	INT				cv / km	INT			
cv / peso	INT				disponible	BOOL			
videoFile									

Imagen.tipo: GALERIA · R360

Reserva.estado: PENDIENTE · ACEPTADA · CANCELADA · RECHAZADA

6 - ROLES DEL SISTEMA

ADMIN	VENDEDOR	MECÁNICO
• Acceso completo a toda la app • Crear mecánicos y vendedores • Ver historial de todos los clientes • Gestionar catálogo, taller y reservas	• Gestionar catálogo de motos nuevas • Subir imágenes, 360° y vídeos • Gestionar motos de segunda mano • Aceptar/rechazar reservas	• Ver y aceptar citas del taller • Crear y enviar presupuestos • Gestionar estados de reparación • Historial de clientes
CLIENTE	INVITADO	
• Registrarse con confirmación de email • Reservar motos de segunda mano • Pedir citas en el taller • Aceptar/rechazar presupuestos	• Consultar catálogo de motos nuevas • Consultar motos de segunda mano • Sin posibilidad de reservar ni pedir citas	

Los permisos se configuran en SecurityConfig.kt mediante reglas por ruta y rol. Los mecánicos y vendedores son creados directamente por el admin sin necesidad de confirmar email.

7 · APP ANDROID — PANTALLAS PRINCIPALES

Login / Registro

Pestañas con animación. Confirmación por email, recuperación de contraseña y acceso como invitado.

Catálogo motos nuevas

Lista de marcas → categorías → lista de modelos. Ficha con galería deslizable, 360° y vídeo.

Segunda mano

Listado con filtro tolerante a errores tipográficos (Levenshtein). Solicitud de reserva.

Taller (cliente)

Lista de citas con estados. Nueva cita con selector de moto, tipo, fecha y hora. Aprobación de presupuestos.

Taller (mecánico)

Citas pendientes con botones de aceptar/rechazar. Detalle con presupuesto editable y envío al cliente.

Perfil

Datos personales editables. Gestión de vehículos propios. Historial de reparaciones por moto.

8 · FUNCIONALIDADES DESTACADAS

Vista 360°

Secuencia de ≥ 8 imágenes ordenadas por ángulo.

Glide precarga todos los frames al cargar el fragment para evitar parpadeos.

Búsqueda tolerante (Levenshtein)

Filtro de motos de segunda mano con tolerancia a errores tipográficos.

Algoritmo de distancia de edición $O(m \cdot n)$ con umbral ≤ 2 errores.

Kasawaki → Kawasaki

Hnda → Honda

Heurísticas adicionales: subcadena, prefijo y comparación palabra a palabra.

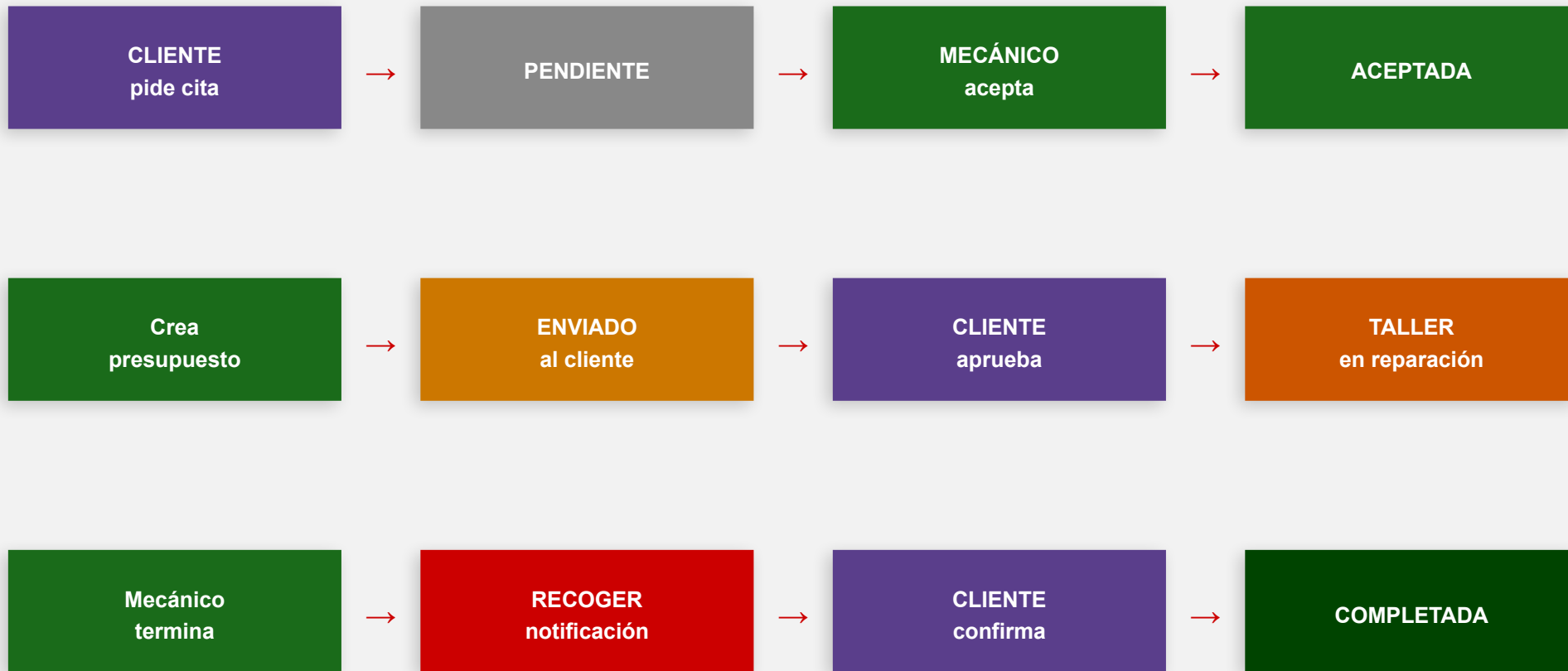
ExoPlayer (Media3)

Vídeos servidos desde el propio servidor. Pantalla completa con extracción del `PlayerView` al `decorView`.

Galería con ViewPager2

Deslizamiento. `GaleriaAdapter` conecta la lista de URLs de imágenes al `ViewPager2` con `Glide` para carga y caché.

FLUJO COMPLETO DEL TALLER



En cualquier punto el mecánico puede rechazar la cita o el cliente puede rechazar el presupuesto, cambiando el estado a RECHAZADA/RECHAZADO.

9 · DESPLIEGUE

Docker Compose + ngrok

docker-compose.yaml

```
services:
  db:
    image: mysql:8.1
    container_name: concesionario-db
    restart: always
    environment: ...
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql

  backend:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: concesionario-api
    restart: always
    ports:
      - "8080:8080"
    environment:
      JWT_SECRET: fF3d6hG7kJ9lM2nP4qR6tS8vX0yZ2aB5cD7eF9gH0iJ1kL2mN3oP4qR6sT8uV0wX
      MYSQL_USER: concesionario
      MYSQL_PASSWORD: concesionario
    depends_on:
      - db
    volumes:
      - ./uploads:/app/uploads
```

backend.yml (ngrok)

```
version: "3"
agent:
  authtoken: 3D8baNWSt7XZVspiD1NtaHVPu9e_2PyYbKUM1dhE9A3tRBjFf
endpoints:
  - name: backend
    url: https://vendor-pasture-scalding.ngrok-free.dev
  upstream:
    url: 8080
```

Comandos

```
docker compose up -d
```

Arrancar backend y base de datos

```
ngrok start backend
```

Activar túnel HTTPS público

```
docker compose build --no-cache
```

Reconstruir imagen desde cero

```
docker compose logs -f backend
```

Ver logs en tiempo real

PROBLEMAS ENCONTRADOS

Modo oscuro

Con DayNight en themes hace que el login y parte de la app se vea con los colores oscuros. Fix cambiamos a Light.

Sesión activa

Android restauraba las SharedPreferences al reinstalar la app, manteniendo la sesión activa después de desinstalar. Fix: Añadir allowBackup="false" en el AndroidManifest.xml

YouTube embebido bloqueado

Los canales oficiales (Yamaha, Honda) bloquean el embebido con error 152/153. Fix: Coger los vídeos desde el propio servidor con ExoPlayer.

Email en el perfil incorrecto

El email del perfil no coincidía con el del registro. Fix: crear el Cliente con el email real en el momento del registro, no con un valor por defecto.

Precarga de fotos 360

Al deslizar en la vista 360° las imágenes tardaban en cargar causando parpadeos. Fix: Precargar todos las fotos con Glide.preload() al entrar en la pantalla.

CG-NAT sin acceso exterior

La conexión doméstica usa CG-NAT e impide abrir puertos. Fix: usar ngrok con dominio fijo para crear un túnel HTTPS sin tocar el router.

OBJETIVOS CONSEGUIDOS

1

Diseñar el modelo de datos: entidades como motocicletas, marcas, clientes, ventas y citas.

2

Implementar una base de datos relacional que garantice la integridad de los datos.

3

Desarrollar un backend con Spring Boot que proporcione una API REST para gestionar la información.

4

Implementar operaciones CRUD para motos, citas, reservas, clientes, vendedores y mecánicos.

5

Desarrollar una aplicación móvil Android que interactúe con la API REST.

6

Integrar la app Android con el backend mediante consumo de servicios web.

7

Implementar funcionalidades de consulta del catálogo, registro de clientes y gestión de ventas.

8

Implementar un sistema de gestión de citas para el taller: nuevas citas de revisión o reparación.

9

Permitir la modificación de citas existentes: fecha, hora u otra información.

10

Permitir la cancelación o eliminación de citas para mantener la agenda actualizada.

11

Gestionar la disponibilidad de mecánicos y permitir asignación en paralelo.

12

Evitar la asignación de citas cuando no haya mecánicos disponibles.

13

Usar contenedores Docker para desplegar la base de datos y facilitar la portabilidad.

14

Implementar búsqueda y filtrado de motocicletas por marca, modelo, precio u otras características.

15

Gestionar imágenes de motocicletas: almacenar y visualizar fotografías desde la app.

16

Creación de usuarios con roles diferenciados: Invitado, Cliente, Administrador, Vendedor y Mecánico.

FUTURAS MEJORAS

Notificaciones push

Firestore Cloud Messaging

Enviar notificaciones reales al dispositivo cuando la moto esté lista, incluso con la app cerrada.

Mapa

Google Maps

Añadir una redirección a google maps sobre la ubicación del concesionario

Panel web de admin

Bootstrap + Javascript

Dashboard web para gestionar usuarios, ver estadísticas del taller y controlar el inventario desde un PC.

CI/CD en producción

GitHub Actions + VPS

Despliegue automático al subir código al repositorio, con ejecución de tests y entrega continua.

CONCLUSIÓN

01

Sistema completo

Backend REST stateless con Spring Boot, base de datos MySQL en Docker y app Android nativa con MVVM.

02

Seguridad real

JWT + BCrypt + confirmación de email + reset de contraseña. Permisos granulares por rol en cada endpoint.

03

UX diferenciada

Vista 360°, galería con deslizamiento, vídeo en fullscreen y búsqueda tolerante a errores tipográficos.

04

5 roles definidos

Admin, Vendedor, Mecánico, Cliente e Invitado con accesos diferenciados tanto en app como en backend.

05

Despliegue simple

Un único docker compose up levanta todo el entorno. ngrok expone el backend con HTTPS sin necesidad de VPS.

KAZ MOTORS

Gracias por vuestra atención

Antonio Jesús Castillo Henares · TFG 2025/26 · IES Gran Capitán · Córdoba

[Repositorio en GitHub](#)